

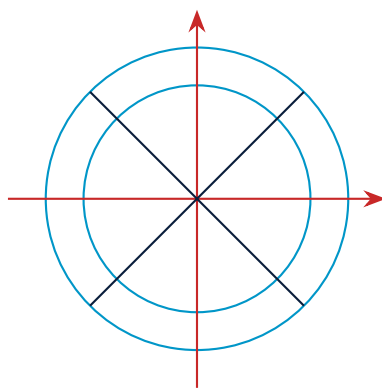
PROJECT SUDARSHAN

Quad-Domain Autonomous C4ISR System

System Architecture & Deep Mathematical Specification

Prepared by: **Eternalcodertanishq3**

Project Guide: **Dr. Prathamesh Potdar**



AIR · LAND · SEA · SPACE

June 14, 2026

Contents

1	Executive Summary	2
2	System Architecture	2
2.1	Architectural Diagram	2
3	Deep-Dive: Engine Mechanics & Algorithms	3
3.1	1. The Vision Engine: Tensor Processing & Perception	3
3.2	2. The Kinematic Engine: Bridging Occlusion with Math	3
3.3	3. The Orbital Engine: ECEF to Topocentric Transformation	4
3.4	4. The Tactical Command Engine: Bayesian Sensor Fusion	4
3.5	5. Local Hardware Execution Strategy	4
4	Technology Stack	5
5	Operational Example: Red Alert Scenario	5

1 Executive Summary

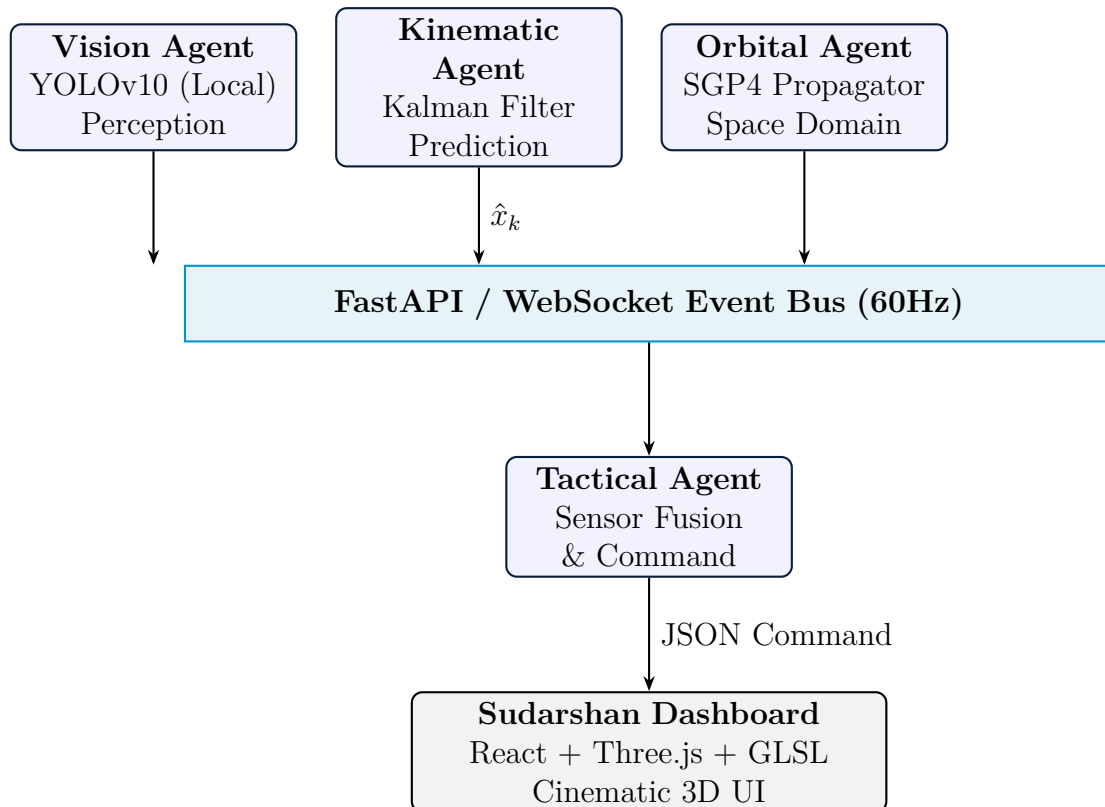
Project Sudarshan represents a paradigm shift in autonomous surveillance. It is an air-gapped, zero-API, quad-domain (Air, Land, Sea, and Space) Command, Control, Communications, Computers, Intelligence, Surveillance, and Reconnaissance (C4ISR) system. Designed to run strictly on local edge-compute environments (such as dedicated Parrot OS security nodes), it ensures absolute operational security (OPSEC).

Unlike conventional hackathon projects relying on cloud-based Large Language Models, Project Sudarshan utilizes deterministic mathematical micro-agents to process computer vision tensors, predict kinematic trajectories, propagate orbital mechanics, and orchestrate tactical responses locally.

2 System Architecture

The architecture rejects the monolithic cloud API model. It operates on a high-frequency event bus orchestrated via FastAPI and WebSockets, facilitating real-time data exchange between four specialized Python micro-agents.

2.1 Architectural Diagram



3 Deep-Dive: Engine Mechanics & Algorithms

A military C4ISR system is only as good as its ability to process chaotic, noisy data and output deterministic, high-confidence commands. The following sections detail the mathematical and operational core of Project Sudarshan.

3.1 1. The Vision Engine: Tensor Processing & Perception

The Vision Agent does not simply "look at pictures." It performs high-speed matrix multiplications to extract semantic meaning from raw pixel arrays.

- **The Ingestion Pipeline:** The local camera feed is captured using OpenCV. Every frame is converted into a multidimensional tensor (e.g., $1 \times 3 \times 640 \times 640$) representing the RGB color channels.
- **The Inference Pass:** This tensor is fed into the local YOLOv10 neural network running on the local GPU via PyTorch. The network applies convolutional filters to identify edges, shapes, and eventually, specific military entities (UAVs, armored vehicles, frigates).
- **Non-Maximum Suppression (NMS):** The model initially guesses hundreds of possible bounding boxes around a single target. NMS mathematically filters these down to the single best box by calculating the Intersection over Union (IoU) of overlapping boxes:

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} \quad (1)$$

If $IoU > 0.45$, the system discards the lower-confidence box, leaving one clean, highly accurate tracking box.

3.2 2. The Kinematic Engine: Bridging Occlusion with Math

Cameras are optical; they fail when lines of sight are broken. If a drone flies behind a mountain, the Vision Agent drops the track. The Kinematic Agent takes over, using an **Extended Kalman Filter (EKF)** to transition from optical tracking to mathematical prediction.

- **The State Vector:** The system defines the target not just by where it is, but by how it is moving. It creates a state matrix $\mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix} \quad (2)$$

- **The Prediction Phase:** When the target vanishes, the EKF uses physics to predict its next location by applying a state transition matrix $\mathbf{F}$. It continuously updates its positional covariance $\mathbf{P}$ (its level of uncertainty):

$$\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^T + \mathbf{Q} \quad (3)$$

- **Re-acquisition:** Because the system mathematically projected the flight path, the camera is already "looking" at the exact pixels where the target will emerge from behind the occlusion, allowing for zero-latency re-acquisition.

3.3 3. The Orbital Engine: ECEF to Topocentric Transformation

The Space Domain operates entirely offline using the `skyfield` Python library and a downloaded CelesTrak Two-Line Element (TLE) catalog.

- **SGP4 Propagation:** The agent feeds the TLE data into the SGP4 (Simplified General Perturbations) algorithm. This calculates the exact X, Y, Z coordinates of a spy satellite in the Earth-Centered, Earth-Fixed (ECEF) coordinate system.
- **Relative Threat Calculation:** The agent mathematically transforms the ECEF coordinates into Topocentric coordinates (Azimuth, Elevation, and Range) relative to the local latitude and longitude of the command base.
- **The Trigger:** If the satellite's Elevation angle exceeds 15° above the horizon, the agent flags an active "Surveillance Pass."

3.4 4. The Tactical Command Engine: Bayesian Sensor Fusion

This is the brain of Project Sudarshan. It takes noisy, imperfect data from the Air, Land, Sea, and Space agents and fuses it to determine the absolute true threat level using Bayesian probability.

If the Vision Agent is only 60% sure it sees a drone (E_V), but the Orbital Agent confirms a spy satellite is directly overhead (E_O), the Tactical Agent updates the overall Threat Probability $P(T)$ using a derived Bayes' formula:

$$P(T|E_V, E_O) = \frac{P(E_V, E_O|T)P(T)}{P(E_V, E_O)} \quad (4)$$

The fusion of multiple weak signals creates one undeniable, high-confidence alert.

3.5 5. Local Hardware Execution Strategy

To ensure the UI doesn't freeze while the math is running, the architecture leverages Python's `multiprocessing` library:

- **Process A (Vision):** Runs the YOLO model as fast as the GPU allows, dumping bounding box coordinates into a shared memory queue.
- **Process B (Kinematics & Orbit):** Reads the queue, runs the Kalman filter and orbital math, and passes the refined data to the Orchestrator.
- **Process C (Event Bus):** Exists solely to grab the final JSON payload via FastAPI and blast it to the React frontend at 60Hz.

This isolated architecture guarantees that even if the camera feed stutters, the orbital tracking and user interface remain perfectly fluid.

4 Technology Stack

- **Core Environment:** Parrot OS (Target deployment environment for max OPSEC)
- **Edge Backend:** Python 3.11+, FastAPI, WebSockets, Python Multiprocessing
- **AI/Math Engine:** PyTorch, Ultralytics YOLOv10, OpenCV, NumPy, SciPy, Skyfield
- **Frontend UI:** React 18, TypeScript, Vite, Tailwind CSS, Zustand
- **Graphics/Rendering:** Three.js (3D Earth/Orbital tracks), Custom GLSL Shaders (Holographic sweeps, claymorphism panels).

5 Operational Example: Red Alert Scenario

T-Minus 00:00 (Space Domain): The Three.js 3D globe displays a red orbital track. The Orbital Agent runs its offline SGP4 calculation and flags a foreign LEO reconnaissance satellite approaching overhead.

T-Minus 00:45 (Air/Land Domain): A ground camera feed activates. The local Vision Agent running YOLOv10 identifies a small, fast-moving pixel with 0.85 confidence. Bounding box coordinates are extracted via tensor inference.

T-Minus 00:48 (Kinematic Tracking): The drone moves behind terrain, causing the Vision confidence to drop to 0.0. The Kinematic Agent instantly applies the Extended Kalman Filter, projecting a holographic vector line predicting its exact emergence point based on state transition matrices.

T-Minus 00:50 (Tactical Response): The Tactical Agent fuses the simultaneous satellite pass and drone incursion. The Bayesian sensor fusion calculation exceeds the critical threshold (0.95). The system autonomously generates interception coordinates and triggers the visual RED ALERT state on the React frontend.